

Enrutamiento adaptativo de modelos de lenguaje: ¿superan las políticas aprendidas a una regla determinista?

Rodolfo Fritz · Benjamín Cerda · Felipe Friz

Universidad del Bío-Bío — Concepción, Chile · Inteligencia Artificial en Automática/Automatización — Proyecto Final 2026

Introducción

Los sistemas basados en modelos de lenguaje (LLM) enfrentan un dilema de ingeniería: los modelos más capaces cuestan **dos órdenes de magnitud más** que los pequeños, pero no toda consulta requiere esa capacidad. Un **router** decide, consulta a consulta, qué modelo ejecutar.

Pregunta: ¿logran las políticas de enrutamiento aprendidas una utilidad esperada superior a una política ponderada determinista al elegir entre modelos heterogéneos?

Objetivo: comparar dos algoritmos de IA — **XGBoost** (supervisado) y **LinUCB** (bandit contextual) — contra una baseline determinista y una cota superior (Oracle), bajo una utilidad que pondera calidad, costo, latencia y error.

Materiales y métodos: datos

RouterBench 0-shot [1]: resultados reales de 11 LLM sobre benchmarks públicos, fijado por SHA-256 y convertido a un esquema canónico auditado (una fila por par consulta-modelo).

36.497

prompts

4

brazos de modelo

145.988

filas limpias

Brazos: **cuatro niveles de costo con calidad creciente**, excluyendo modelos dominados — Mistral 7B (rápido), Mixtral 8x7B (equilibrado), Yi-34B (razonamiento), GPT-4 Turbo (premium).

Preprocesamiento: limpieza determinista (duplicados, rangos, consistencia por prompt); características *pre-inferencia* (longitud del texto, tipo de tarea one-hot); partición **70/15/15 agrupada por prompt** y estratificada por tarea, con 5 semillas bloqueadas y pruebas automáticas contra fugas de información.

Función de utilidad

Definida y bloqueada antes de ver resultados:

$$U = 0.65 \cdot Q - 0.20 \cdot C_n - 0.10 \cdot L_n - 0.05 \cdot E$$

Q: calidad $\in [0,1]$; C_n , L_n : costo y latencia min-max **anclados al split de train**; E: indicador de error. La latencia del artefacto es nula y su término queda deshabilitado explícitamente.

Consulta (prompt)



Características pre-inferencia (longitud, tipo de tarea)



Reglas

(baseline)

XGBoost

LinUCB

Oracle



Modelo seleccionado → calidad, costo, error → utilidad U

Algoritmos y evaluación

- Better Rules Proxy:** tabla determinista tarea→brazo con mejor utilidad media en train.
- XGBoost** [3]: un regresor de utilidad por brazo; ruteo por argmax; grilla de 8 configuraciones elegida *solo en validación*.
- LinUCB disjunto** [2]: bandit contextual entrenado por *replay prequential* (elegir→observar→actualizar) sobre un barajado con semilla de train; α elegido en validación.
- Oracle offline:** mejor brazo realizado por prompt (cota superior no desplegable).

Métricas: utilidad media, calidad media, costo medio y tasa de error en test; **IC 95 % por bootstrap pareado** (2000 remuestras por *prompt*) sobre 5 semillas.

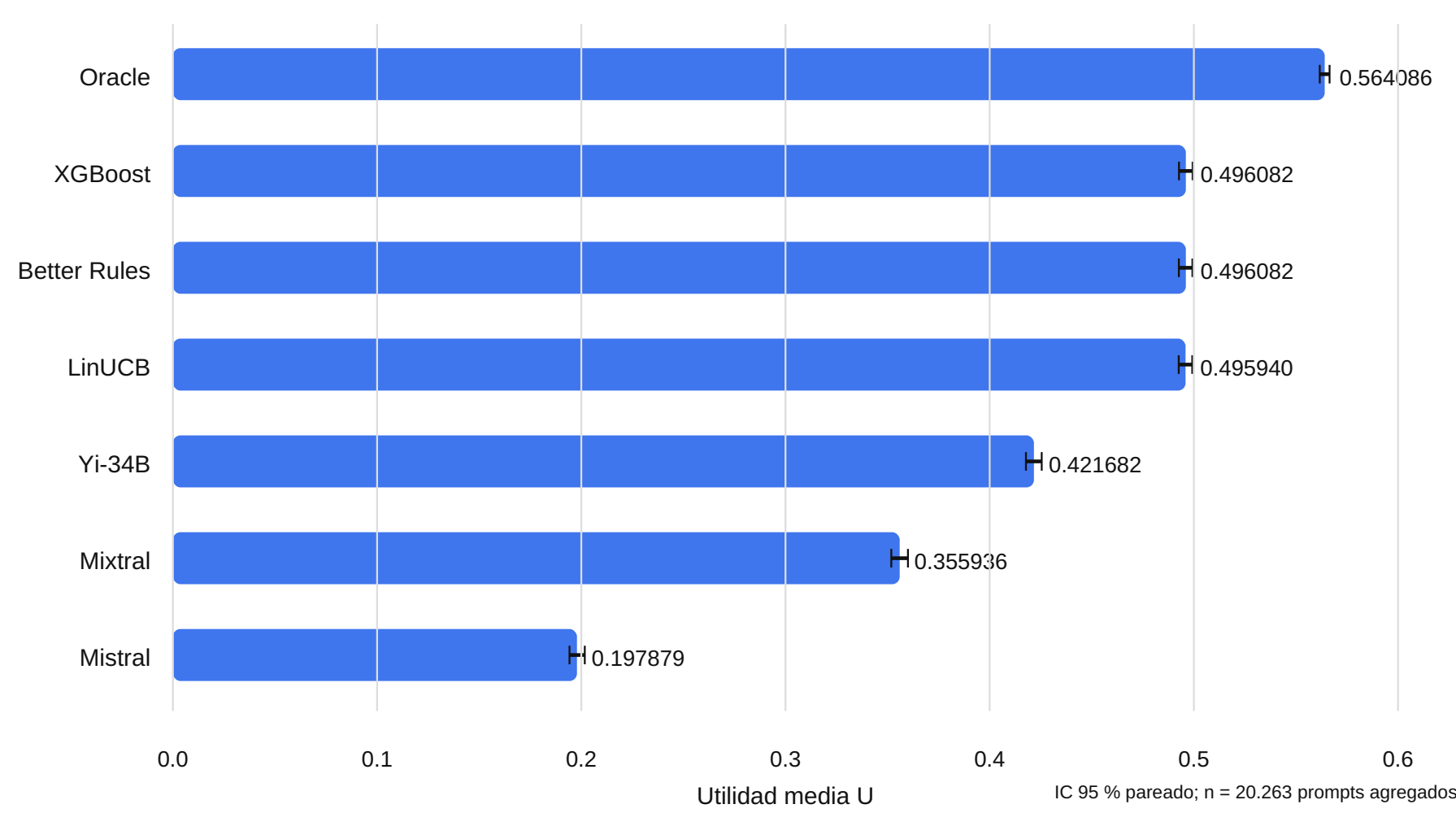
Implementación: Python modular (pandas, scikit-learn, xgboost, numpy), CI con mypy estricto, ruff y cobertura ≥ 85 %; tablas y figuras generadas desde artefactos, nunca a mano.

Resultados

Política	Utilidad U	Δ vs baseline [IC 95 %]	Calidad	Costo [USD]
Oracle offline (cota superior)	0.5641	+0.0680 [+0.0656, +0.0705]	0.871	0.00061
Better Rules Proxy (baseline)	0.4961	—	0.783	0.00329
Router XGBoost	0.4961	+0.0000 [-0.0005, +0.0006]	0.783	0.00317
Router LinUCB	0.4959	-0.0001 [-0.0005, +0.0002]	0.783	0.00325
Fijo: GPT-4 Turbo	0.4961	+0.0000 [+0.0000, +0.0000]	0.783	0.00329
Fijo: Yi-34B	0.4217	-0.0744 [-0.0783, -0.0704]	0.649	0.00019
Fijo: Mixtral 8x7B	0.3559	-0.1401 [-0.1445, -0.1358]	0.548	0.00013
Fijo: Mistral 7B	0.1979	-0.2982 [-0.3031, -0.2932]	0.302	0.00005

Media sobre los splits de prueba de 5 semillas (20.263 prompts únicos). Δ : diferencia pareada contra Better Rules Proxy; IC 95 % bootstrap por prompt.

Utilidad media por política — RouterBench 0-shot

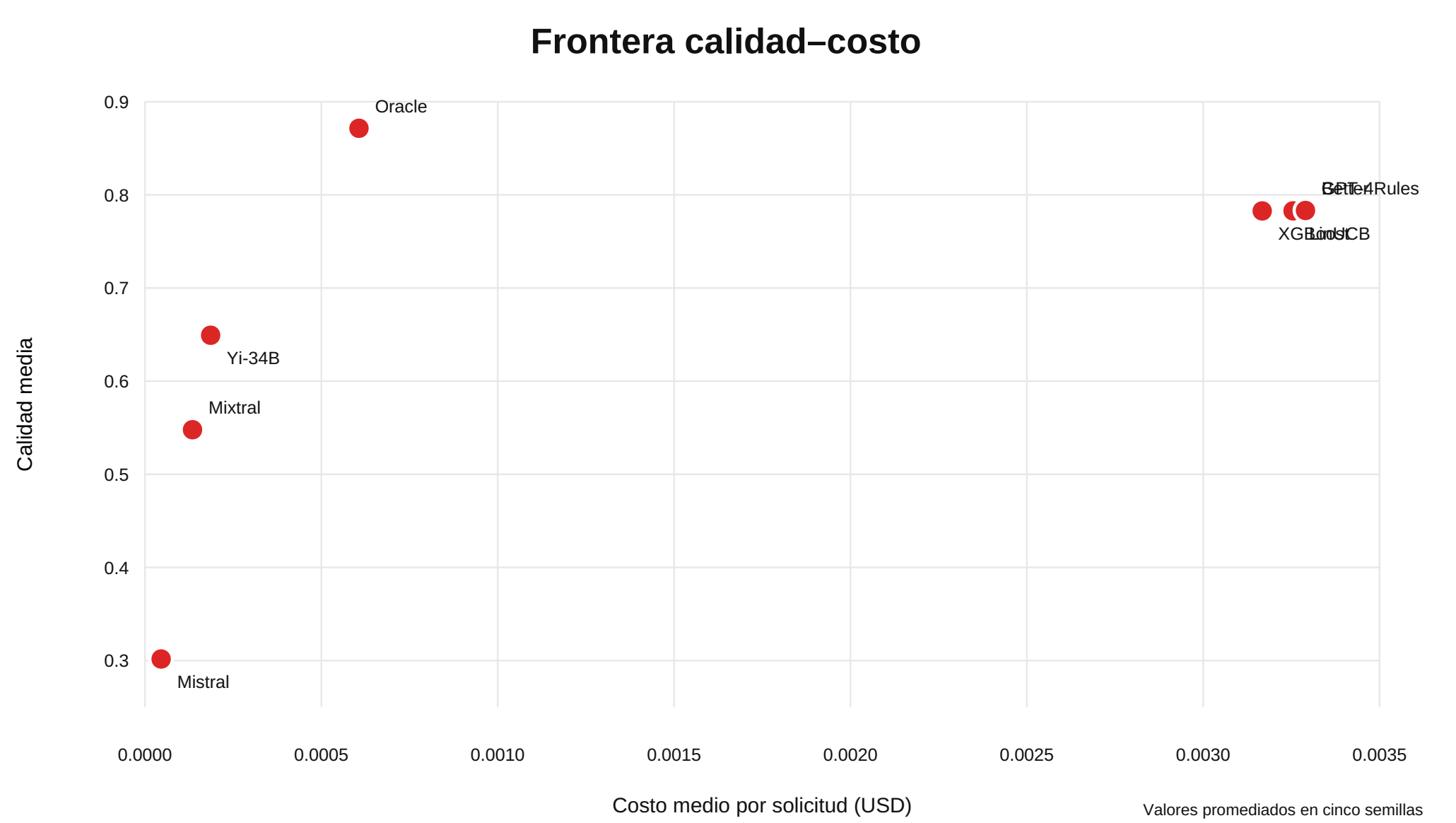


Utilidad media en test con IC 95 %. XGBoost y LinUCB **igualan** a la baseline (los IC de las diferencias cruzan cero); solo el Oracle la supera (+0.068).

Conclusiones

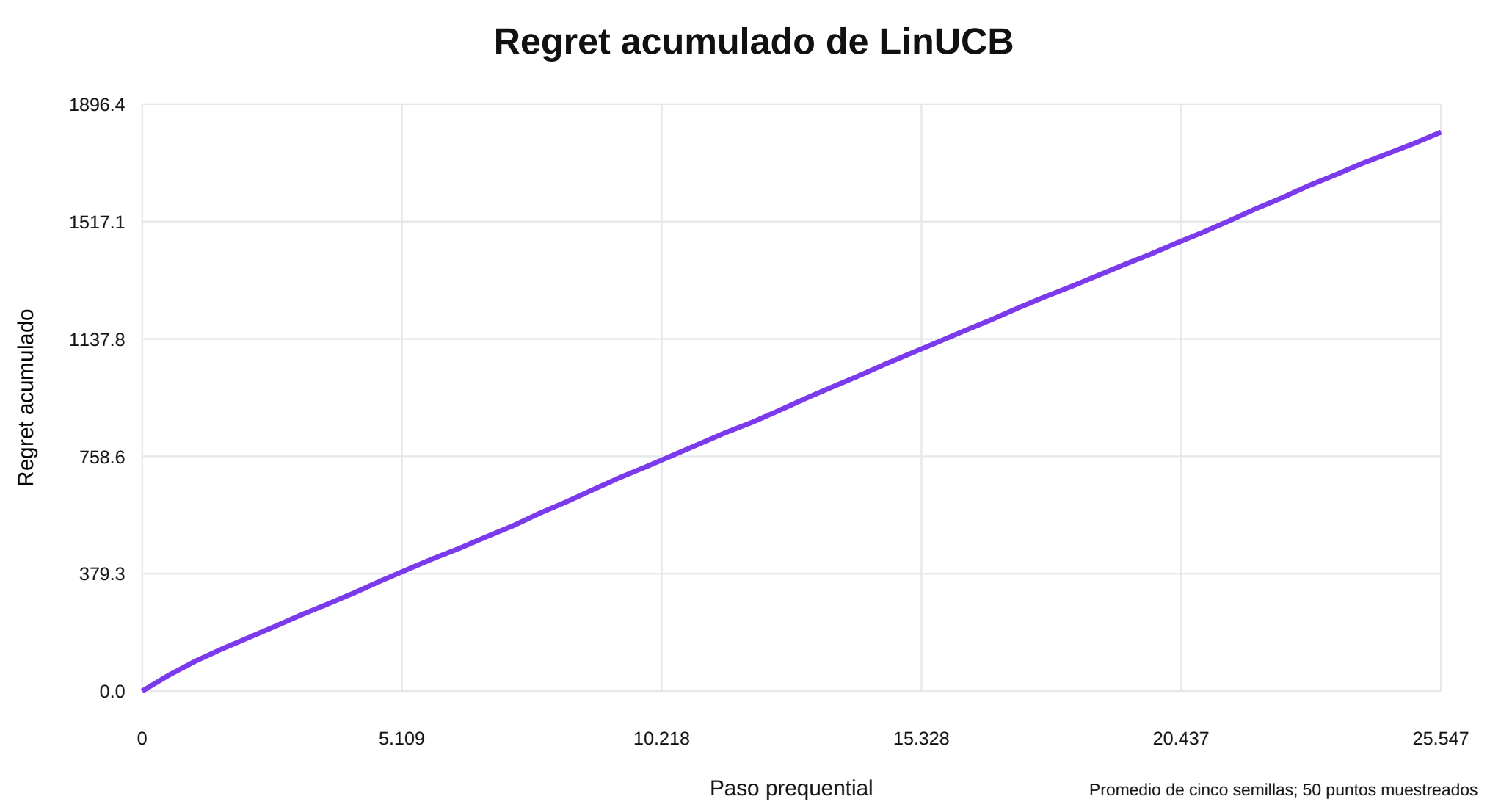
- Con características pre-inferencia simples, **las políticas aprendidas igualan pero no superan** a la regla determinista: $\Delta \approx 0$ con IC 95 % que cruza cero.
- Bajo los pesos elegidos, la baseline converge a «siempre GPT-4»; el problema resulta trivial para las señales disponibles.
- El Oracle (+0.068 de utilidad, mejor calidad a $\approx 5\times$ menos costo) demuestra que **el potencial del ruteo por consulta es real** y queda sin capturar.
- Limitaciones:** características simples, latencia ausente, pesos fijos y selección exploratoria de los cuatro brazos.
- Trabajo futuro:** embeddings del prompt como contexto, análisis de sensibilidad de pesos y evaluación en modo sombra.

Frontera calidad-costo



El Oracle logra **más calidad (0.87 vs 0.78) a $\approx 5\times$ menos costo** que «siempre GPT-4»: el margen del ruteo por consulta existe, pero exige mejores señales.

LinUCB: regret acumulado



Regret acumulado durante el replay prequential (5 semillas): crecimiento estable y reproducible, sin exploración catastrófica.

Referencias

- Q. J. Hu *et al.*, «RouterBench: A Benchmark for Multi-LLM Routing Systems», arXiv: 2403.12031, 2024. DOI dataset: 10.57967/hf/1996.
- L. Li, W. Chu, J. Langford y R. E. Schapire, «A Contextual-Bandit Approach to Personalized News Article Recommendation», WWW, 2010.
- T. Chen y C. Guestrin, «XGBoost: A Scalable Tree Boosting System», KDD, 2016.
- Código del estudio: github.com/FitoFritzG/better-router-adaptive-research (MIT).